

JBackground

Documentacion Tecnica
Componente Personalizado de Java Swing

Version	1.0
Lenguaje	Java (Swing)
Paquete	<code>javax.swing</code> compatible
IDE	NetBeans (compatible con <code>AbsoluteLayout</code>)

Este documento describe todas las clases, campos, constructores, metodos y clases internas del componente **JBackground**.

Índice de Contenidos

1	Descripcion General	3
1.1	Objetivos de Diseno	3
1.2	Jerarquia de Clases	3
1.3	Dependencias	3
2	Campos	4
2.1	currentImage	4
2.2	listeners	4
2.3	fileChooser	4
2.4	overlayPanel	4
3	Constructores	4
3.1	JBackground()	4
3.2	JBackground(String text)	5
4	Metodos Publicos	5
4.1	getCurrentImage()	5
4.2	setBackgroundImage(BufferedImage image)	5
4.3	clearBackground()	5
4.4	addBackgroundChangeListener(BackgroundChangeListener l)	6
4.5	removeBackgroundChangeListener(BackgroundChangeListener l) . .	6
5	Metodos Privados	6
5.1	handleClick()	6
5.2	applyToFrame(BufferedImage image)	7
5.3	syncOverlaySize(JLayeredPane layeredPane)	7
5.4	findParentFrame()	8
5.5	fireListeners(BufferedImage image)	8
5.6	styleButton()	8
5.7	buildFileChooser() (estatico)	8
6	Interfaz Interna: BackgroundChangeListener	9
6.1	backgroundChanged(BufferedImage image)	9
7	Clase Interna: ImageOverlayPanel	9
7.1	Campos	9
7.1.1	image	9
7.2	Constructor	9
7.3	Metodos	10
7.3.1	setImage(BufferedImage image)	10
7.3.2	paintComponent(Graphics g)	10
8	Guia de Uso	10
8.1	Uso Basico en NetBeans	10
8.2	Escuchar Cambios de Fondo	11
8.3	Establecer un Fondo Predeterminado porCodigo	11
8.4	Limpiar el Fondo	11

9	Limitaciones Conocidas	11
10	Resumen Completo de la Clase	12

1 Descripcion General

JBackground es un componente personalizado de Java Swing que extiende `javax.swing.JButton`. Al hacer clic en el boton, se abre un dialogo nativo de seleccion de archivos para que el usuario elija una imagen. La imagen seleccionada se aplica como fondo del `JFrame` padre, inyectando un panel de superposicion transparente en el `JLayeredPane` del marco en la capa de renderizado mas baja.

1.1 Objetivos de Diseno

- **Facilidad de uso** — se comporta como cualquier `JButton` estandar; no requiere configuracion especial en el formulario anfitrión.
- **Independencia del layout** — funciona con cualquier gestor de diseno, incluido el `AbsoluteLayout` de NetBeans.
- **No destructivo** — los componentes ya presentes en el formulario nunca se mueven, eliminan ni ocultan.
- **Orden Z correcto** — la imagen de fondo siempre se renderiza *por debajo* de todos los demas componentes.
- **Extensible** — una interfaz de escucha de eventos permite que el codigo externo reaccione cuando cambia el fondo.

1.2 Jerarquia de Clases

```

    java.lang.Object
    → java.awt.Component
    → java.awt.Container
    → javax.swing.JComponent
    → javax.swing.AbstractButton
      → javax.swing.JButton
        → JBackground
  
```

1.3 Dependencias

JBackground utiliza unicamente clases de la Biblioteca Estandar de Java. No se requieren bibliotecas externas.

Importacion	Proposito
<code>javax.swing.*</code>	Kit de herramientas Swing
<code>javax.swing.filechooser.FileNameExtensionFilter</code>	Filtro de archivos de imagen
<code>java.awt.*</code>	Clases base de AWT
<code>java.awt.event.*</code>	Escuchas de eventos
<code>java.awt.image.BufferedImage</code>	Imagen en memoria
<code>java.io.File</code>	Referencia a archivo
<code>java.io.IOException</code>	Manejo de errores de E/S
<code>javax.imageio.ImageIO</code>	Lectura de imagenes

2 Campos

2.1 `currentImage`

```
1 private BufferedImage currentImage;
```

Almacena la imagen de fondo seleccionada mas recientemente. Inicialmente `null`. Se actualiza cada vez que el usuario selecciona correctamente una imagen a traves del selector de archivos o llama a `setBackgroundImage()`.

2.2 `listeners`

```
1 private final java.util.List<BackgroundChangeListener>
   listeners
2     = new java.util.ArrayList<>();
```

Lista de objetos `BackgroundChangeListener` registrados. Todos los escuchas son notificados (en orden de registro) cada vez que la imagen de fondo cambia correctamente.

2.3 `fileChooser`

```
1 private final JFileChooser fileChooser = buildFileChooser();
```

Una unica instancia compartida de `JFileChooser`, inicializada una sola vez mediante el metodo auxiliar privado `buildFileChooser()`. Reutilizar la instancia preserva el ultimo directorio visitado entre clics.

2.4 `overlayPanel`

```
1 private ImageOverlayPanel overlayPanel;
```

Instancia creada de forma diferida de la clase interna `ImageOverlayPanel`. Se inserta en el `JLayeredPane` del `JFrame` padre la primera vez que se aplica una imagen y se reutiliza en todas las actualizaciones posteriores. Inicialmente `null`.

3 Constructores

3.1 `JBackground()`

```
1 public JBackground()
```

Descripcion: Constructor sin argumentos. Crea un boton `JBackground` con la etiqueta predeterminada "Set Background".

Delega en: `JBackground(String text)` con "Set Background".

3.2 JBackground(String text)

```
1 public JBackground(String text)
```

Descripcion: Constructor principal. Establece la etiqueta del boton como `text`, aplica el estilo visual incorporado y registra el `ActionListener` interno que llama a `handleClick()` cuando se presiona el boton.

Parametro	Tipo	Descripcion
<code>text</code>	<code>String</code>	Etiqueta mostrada en la cara del boton

4 Metodos Publicos

4.1 getCurrentImage()

```
1 public BufferedImage getCurrentImage()
```

Retorna: El `BufferedImage` actualmente configurado como fondo, o `null` si aun no se ha seleccionado ninguna imagen.

Seguridad de hilos: Debe llamarse en el hilo de despacho de eventos (EDT).

4.2 setBackgroundImage(BufferedImage image)

```
1 public void setBackgroundImage(BufferedImage image)
```

Descripcion: Establece una imagen de fondo de forma programatica sin abrir el dialogo de seleccion de archivos. Util para aplicar un fondo predeterminado al iniciar la aplicacion o restaurar una imagen guardada previamente.

Parametro	Tipo	Descripcion
<code>image</code>	<code>BufferedImage</code>	La imagen a aplicar como fondo

Efectos secundarios:

1. Actualiza `currentImage`.
2. Llama a `applyToFrame(image)`.
3. Notifica a todos los `BackgroundChangeListener` registrados.

4.3 clearBackground()

```
1 public void clearBackground()
```

Descripcion: Elimina la imagen de fondo actual. Establece `currentImage` en `null` y, si existe un `overlayPanel`, borra su imagen y activa un repintado. El panel de superposicion permanece en el `JLayeredPane` pero no pinta nada.

4.4 addBackgroundChangeListener(BackgroundChangeListener l)

```
1 public void  
   addBackgroundChangeListener(BackgroundChangeListener l)
```

Descripcion: Registra un escucha para ser notificado cada vez que cambie la imagen de fondo. Los valores nulos se ignoran silenciosamente.

Parametro	Tipo	Descripcion
1	BackgroundChangeListener	Escucha a registrar

4.5 removeBackgroundChangeListener(BackgroundChangeListener l)

```
1 public void  
   removeBackgroundChangeListener(BackgroundChangeListener l)
```

Descripcion: Elimina el registro de un escucha previamente agregado. Si el escucha no esta en la lista, este metodo no tiene efecto.

Parametro	Tipo	Descripcion
1	BackgroundChangeListener	Escucha a eliminar

5 Metodos Privados

5.1 handleClick()

```
1 private void handleClick()
```

Descripcion: Invocado por el `ActionListener` interno cada vez que se hace clic en el boton. Abre el dialogo del selector de archivos. Si el usuario aprueba un archivo, lo lee con `ImageIO.read()`. En caso de exito llama a `applyToFrame()` y `fireListeners()`. En caso de fallo muestra un dialogo de error apropiado.

Flujo de ejecucion:

1. Localizar el `Window` ancestro mediante `SwingUtilities.getWindowAncestor()`.
2. Mostrar `fileChooser.showOpenDialog()`.
3. Si el resultado $\neq$ `APPROVE_OPTION`, retornar inmediatamente.
4. Llamar a `ImageIO.read(file)`.
5. Si el resultado es `null` (formato no soportado), mostrar un dialogo de advertencia y retornar.
6. Ante `IOException`, mostrar un dialogo de error y retornar.
7. De lo contrario, actualizar `currentImage`, llamar a `applyToFrame()` y a `fireListeners()`.

5.2 applyToFrame(BufferedImage image)

```
1 private void applyToFrame(BufferedImage image)
```

Descripcion: La rutina central de aplicacion de fondo. Inyecta (o actualiza) un `ImageOverlayPanel` en el `JLayeredPane` del `JFrame` padre en un indice de capa inferior a `JLayeredPane.FRAME_CONTENT_LAYER`, garantizando que la superposicion siempre se renderice detras de todos los demas componentes.

Parametro	Tipo	Descripcion
image	<code>BufferedImage</code>	La imagen a pintar como fondo

Pasos detallados:

1. Llamar a `findParentFrame()` — si es `null`, abortar.
2. Obtener referencias al `JLayeredPane` y al `contentPane` del marco.
3. **Solo en la primera llamada:** instanciar `overlayPanel`, agregarlo al `layeredPane` en la capa `FRAME_CONTENT_LAYER - 1` y adjuntar un `ComponentListener` para mantener el tamaño de la superposicion ajustado al panel en capas.
4. Actualizar la imagen de `overlayPanel` y sincronizar sus limites con `syncOverlaySize()`.
5. Llamar a `contentPane.setOpaque(false)` para que la superposicion sea visible a traves de el.
6. Usar `SwingUtilities.invokeLater()` para programar `revalidate() + repaint()` tanto en el panel en capas como en el panel de contenido, asegurando que todos los componentes se redibuje encima del nuevo fondo.

Por que invokeLater? El repintado se difiere al *siguiente* ciclo del EDT para que Swing termine de procesar el evento actual antes de redibujar. Esto evita que los componentes desaparezcan brevemente tras aplicar la imagen.

5.3 syncOverlaySize(JLayeredPane layeredPane)

```
1 private void syncOverlaySize(JLayeredPane layeredPane)
```

Descripcion: Establece los limites del `overlayPanel` para que coincidan con el tamaño completo del `layeredPane` (`x=0`, `y=0`, `ancho`, `alto`). Se llama tanto cuando se crea la superposicion por primera vez como desde el `ComponentListener` cada vez que se redimensiona el marco.

Parametro	Tipo	Descripcion
layeredPane	<code>JLayeredPane</code>	El panel cuyas dimensiones deben igualarse

5.4 findParentFrame()

```
1 private JFrame findParentFrame()
```

Descripcion: Recorre la jerarquia de componentes para localizar el JFrame anfitrión. Primero intenta con `SwingUtilities.getWindowAncestor(this)`. Si no es un JFrame (por ejemplo, durante el tiempo de diseño en NetBeans), itera todas las Windows abiertas y retorna el primer JFrame visible.

Retorna: El JFrame padre, o null si no se encuentra ninguno.

5.5 fireListeners(BufferedImage image)

```
1 private void fireListeners(BufferedImage image)
```

Descripcion: Itera sobre listeners y llama a `backgroundChanged(image)` en cada uno.

Parametro	Tipo	Descripcion
image	BufferedImage	La imagen que fue aplicada

5.6 styleButton()

```
1 private void styleButton()
```

Descripcion: Aplica un estilo visual al boton: fondo azul, texto blanco, un borde compuesto, cursor de mano y efecto hover implementado mediante un `MouseAdapter` anonimo. Se llama una vez desde el constructor.

Propiedades de estilo aplicadas:

Propiedad	Valor
Fondo (normal)	RGB(55, 110, 195)
Fondo (hover)	RGB(35, 85, 165)
Primer plano	Color.WHITE
Fuente	SansSerif, NEGRITA, 13pt
Borde	linea de 1px + relleno vacio de 6/14px
Cursor	HAND_CURSOR
Foco pintado	false

5.7 buildFileChooser() (estatico)

```
1 private static JFileChooser buildFileChooser()
```

Descripcion: Metodo de fabrica que construye y configura el `JFileChooser` compartido. Deshabilita el filtro “Todos los archivos” y agrega un unico filtro que acepta formatos de imagen comunes.

Extensiones aceptadas: .jpg, .jpeg, .png, .gif, .bmp

Retorna: Una instancia de JFileChooser lista para usar.

6 Interfaz Interna: BackgroundChangeListener

```
1 public interface BackgroundChangeListener {
2     void backgroundChanged(BufferedImage image);
3 }
```

Descripcion: Interfaz funcional (SAM) que permite que el codigo externo sea notificado cada vez que la imagen de fondo cambie correctamente, ya sea mediante interaccion del usuario o una llamada programatica a `setBackgroundImage()`.

6.1 backgroundChanged(BufferedImage image)

Parametro	Tipo	Descripcion
image	BufferedImage	La nueva imagen de fondo aplicada

Ejemplo de uso:

```
1 jBackground1.addBackgroundChangeListener(img -> {
2     System.out.println("Fondo cambiado! Tamano: "
3         + img.getWidth() + "x" + img.getHeight());
4 });
```

7 Clase Interna: ImageOverlayPanel

```
1 static class ImageOverlayPanel extends JPanel
```

Descripcion: Subclase de JPanel que pinta un `BufferedImage` escalado para llenar sus limites completos usando escalado *COVER* (preserva la relacion de aspecto, puede recortar). Se inserta en el `JLayeredPane` en la capa mas baja para que siempre se renderice detras de todos los demas componentes.

7.1 Campos

7.1.1 image

```
1 private BufferedImage image;
```

La imagen que se esta pintando actualmente. Puede ser `null`, en cuyo caso `paintComponent` retorna inmediatamente despues de llamar a `super.paintComponent`.

7.2 Constructor

```
1 ImageOverlayPanel()
```

Descripcion: Constructor de acceso de paquete. Establece `opaque = true` para que el panel cubra completamente el area detras de el con la imagen, evitando que el borrado de fondo de Swing deje artefactos.

Por que `setOpaque(true)`? Si el panel no es opaco, Swing puede omitir su pintado durante ciertos ciclos de repintado, haciendo que el contenido antiguo se filtre y que los componentes parezcan desaparecer hasta que el raton pase sobre ellos.

7.3 Metodos

7.3.1 `setImage(BufferedImage image)`

```
1 void setImage(BufferedImage image)
```

Reemplaza la imagen mostrada. Pasar `null` detiene el pintado de cualquier imagen. No llama a `repaint()` — el llamador es responsable de activar un repintado.

7.3.2 `paintComponent(Graphics g)`

```
1 @Override
2 protected void paintComponent(Graphics g)
```

Descripcion: Sobreescribe `JPanel.paintComponent` para dibujar la imagen de fondo. Realiza escalado *COVER*: la imagen se escala para que su dimension menor llene exactamente el panel y se centra. Esto puede recortar la imagen en la dimension mayor.

Sugerencias de renderizado aplicadas:

Sugerencia	Valor
KEY_INTERPOLATION	VALUE_INTERPOLATION_BILINEAR
KEY_RENDERING	VALUE_RENDER_QUALITY

Algoritmo de escalado COVER:

$$\text{escala} = \max\left(\frac{p_a}{i_a}, \frac{p_h}{i_h}\right)$$

$$d_a = i_a \times \text{escala}, \quad d_h = i_h \times \text{escala}, \quad d_x = \frac{p_a - d_a}{2}, \quad d_y = \frac{p_h - d_h}{2}$$

donde p_a, p_h son las dimensiones del panel y i_a, i_h son las dimensiones de la imagen.

8 Guia de Uso

8.1 Uso Basico en NetBeans

1. Compilar `JBackground.java` y agregarlo a la paleta de NetBeans.

2. Arrastrar JBackground al formulario desde la paleta — aparece como un boton normal.
3. En el controlador jBackground1ActionPerformed generado automaticamente, **dejar el cuerpo vacio**:

```

1 private void jBackground1ActionPerformed(ActionEvent evt) {
2     // Dejar vacio --- JBackground maneja los clics
3     // internamente
4 }

```

4. Ejecutar el formulario y hacer clic en el boton para elegir una imagen.

Consejo: No es necesario escribir ningun codigo en el controlador de acciones. El propio ActionListener interno del componente (registrado en el constructor) se activa primero y realiza todo el trabajo.

8.2 Escuchar Cambios de Fondo

```

1 jBackground1.addBackgroundChangeListener(img -> {
2     System.out.println("Nueva imagen: "
3         + img.getWidth() + "x" + img.getHeight());
4     // ej. guardar la referencia de la imagen, actualizar la
5     // barra de estado, etc.
6 });

```

8.3 Establecer un Fondo Predeterminado porCodigo

```

1 // En el constructor del formulario, despues de
2 // initComponents():
3 try {
4     BufferedImage fondoPredeterminado = ImageIO.read(
5         getClass().getResourceAsStream("/imagenes/fondo_default.jpg"));
6     jBackground1.setBackgroundImage(fondoPredeterminado);
7 } catch (IOException e) {
8     e.printStackTrace();
9 }

```

8.4 Limpiar el Fondo

```

1 jBackground1.clearBackground();

```

9 Limitaciones Conocidas

- **Solo JFrame** — el componente busca un ancestro JFrame. No aplicara un fondo a un JDialog o JWindow.
- **Un solo fondo por marco** — todas las instancias de JBackground en el mismo marco comparten un unico ImageOverlayPanel; la ultima imagen aplicada prevalece.

- **Componentes hijos opacos** — si los componentes hijos (por ejemplo, contenedores JPanel) tienen `setOpaque(true)`, pintaran sobre el fondo en su area. Configurarlos como no opacos si se desea que el fondo se muestre a traves de ellos.
- **Look and Feel** — algunos estilos visuales fuerzan que los componentes sean opacos, lo que puede impedir que el fondo se muestre a traves de ciertos controles.

10 Resumen Completo de la Clase

Miembro	Acceso	Descripcion
<code>currentImage</code>	private	Imagen de fondo actualmente aplicada
<code>listeners</code>	private	Lista de escuchas de cambio
<code>fileChooser</code>	private	Dialogo selector de archivos compartido
<code>overlayPanel</code>	private	Panel de superposicion creado de forma diferida
<code>JBackground()</code>	public	Constructor predeterminado
<code>JBackground(String)</code>	public	Constructor con etiqueta personalizada
<code>getCurrentImage()</code>	public	Retorna la imagen actual
<code>setBackgroundImage(BufferedImage)</code>	public	Establece imagen por codigo
<code>clearBackground()</code>	public	Elimina la imagen de fondo
<code>addBackgroundChangeListener()</code>	public	Registra un escucha de cambio
<code>removeBackgroundChangeListener()</code>	public	Elimina un escucha de cambio
<code>handleClick()</code>	private	Logica del selector y carga de imagen
<code>applyToFrame(BufferedImage)</code>	private	Inyecta superposicion en JLayeredPane
<code>syncOverlaySize(JLayeredPane)</code>	private	Mantiene el tamano del overlay
<code>findParentFrame()</code>	private	Localiza el JFrame ancestro
<code>fireListeners(BufferedImage)</code>	private	Notifica a todos los escuchas
<code>styleButton()</code>	private	Aplica estilo visual al boton
<code>buildFileChooser()</code>	private static	Crea el JFileChooser
<code>BackgroundChangeListener</code>	interfaz publica	Escucha de eventos de cambio de imagen
<code>backgroundChanged(BufferedImage)</code>		Llamado cuando cambia la imagen
<code>ImageOverlayPanel</code>	clase estatica	Pinta la imagen en el JLayeredPane
<code>setImage(BufferedImage)</code>	paquete	Actualiza la imagen mostrada
<code>paintComponent(Graphics)</code>	protected	Renderiza imagen con escalado COVER